

Session 19 (AIML) - Assignment Question

- **Name of Student** : Shruti Dnyaneshwar Darwatkar
- **Domain Name**: AIML Development
- **College Name**: Zeal Polytechnic , Narhe
- **Branch Name**: AIML
- **Github url**: https://github.com/shrutidarwatkar-sketch/Task_ITR.git

Q1. Find a dataset that has at least one continuous target variable (e.g., Price, Salary, Sales, Temperature, House Price, Car Mileage, etc.).

Answer:

Dataset Name: Laptop Price Dataset

Continuous Target Variable: Price

The dataset contains laptop specifications like Company, RAM, Processor, Storage, GPU, Operating System, etc. The target variable **Price** is continuous, making this dataset suitable for regression.

Q2. Dataset Selection & Initial Exploration)

- Load the dataset and show first 10 rows.
- Write 56 lines explaining why you chose this dataset and what continuous value you want to predict.

Answer:

- It contains real laptop specifications.
- Price is a continuous numerical value.
- It has both categorical and numerical features.
- It is suitable for regression analysis.

- It helps predict laptop prices based on hardware specifications.

PROGRAM SCREENSHOT :

```
import pandas as pd
```

```
from google.colab import files
uploaded = files.upload()
```

... Choose files: laptop_data.csv
laptop_data.csv(text/csv) - 182053 bytes, last modified: 15/07/2026 - 100% done
Saving laptop_data.csv to laptop_data.csv

```
#Q2

# Display first 10 rows
print(df.head(10))

# Shape
print("Shape:", df.shape)

# Columns
print(df.columns)
```

```

...      Unnamed: 0 Company   TypeName  Inches      ScreenResolution \
0         0      Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
1         1      Apple  Ultrabook   13.3                        1440x900
2         2         HP   Notebook   15.6                        Full HD 1920x1080
3         3      Apple  Ultrabook   15.4  IPS Panel Retina Display 2880x1800
4         4      Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
5         5        Acer  Notebook   15.6                        1366x768
6         6      Apple  Ultrabook   15.4  IPS Panel Retina Display 2880x1800
7         7      Apple  Ultrabook   13.3                        1440x900
8         8        Asus  Ultrabook   14.0                        Full HD 1920x1080
9         9        Acer  Ultrabook   14.0      IPS Panel Full HD 1920x1080

      Cpu  Ram      Memory \
0      Intel Core i5 2.3GHz 8GB      128GB SSD
1      Intel Core i5 1.8GHz 8GB      128GB Flash Storage
2      Intel Core i5 7200U 2.5GHz 8GB      256GB SSD
3      Intel Core i7 2.7GHz 16GB      512GB SSD
4      Intel Core i5 3.1GHz 8GB      256GB SSD
5      AMD A9-Series 9420 3GHz 4GB      500GB HDD
6      Intel Core i7 2.2GHz 16GB      256GB Flash Storage
7      Intel Core i5 1.8GHz 8GB      256GB Flash Storage
8      Intel Core i7 8550U 1.8GHz 16GB      512GB SSD
9      Intel Core i5 8250U 1.6GHz 8GB      256GB SSD

      Gpu      OpSys  Weight      Price
0      Intel Iris Plus Graphics 640      macOS  1.37kg  71378.6832
1      Intel HD Graphics 6000      macOS  1.34kg  47895.5232
2      Intel HD Graphics 620      No OS  1.86kg  30636.0000
3      AMD Radeon Pro 455      macOS  1.83kg  135195.3360
4      Intel Iris Plus Graphics 650      macOS  1.37kg  96095.8080
5      AMD Radeon R5      Windows 10  2.1kg  21312.0000
6      Intel Iris Pro Graphics      Mac OS X  2.04kg  114017.6016
7      Intel HD Graphics 6000      macOS  1.34kg  61735.5360
8      Nvidia GeForce MX150      Windows 10  1.3kg  79653.6000
9      Intel UHD Graphics 620      Windows 10  1.6kg  41025.6000
Shape: (1303, 12)
Index(['Unnamed: 0', 'Company', 'TypeName', 'Inches', 'ScreenResolution',
      'Cpu', 'Ram', 'Memory', 'Gpu', 'OpSys', 'Weight', 'Price'],
      dtype='object')

```

Q3. Missing & Duplicate Values Analysis)

- Check for missing values in each column and calculate their percentage.
- Handle missing values appropriately (drop or fill).
- Check and remove duplicate rows if any.
- Write your observations and steps taken.

Answer:

- No missing values were found.
- Missing percentage is 0% for all columns.

- No duplicate rows were found.
- Therefore, no cleaning was required.

PROGRAM SCREENSHOT :

```
#Q3
# Missing values
missing = df.isnull().sum()

missing_percent = (missing/len(df))*100

print(pd.DataFrame({
    "Missing Values":missing,
    "Percentage":missing_percent
}))

# Duplicate rows
print("Duplicate Rows:",df.duplicated().sum())

# Remove duplicates
df.drop_duplicates(inplace=True)
```

	Missing Values	Percentage
Unnamed: 0	0	0.0
Company	0	0.0
TypeName	0	0.0
Inches	0	0.0
ScreenResolution	0	0.0
Cpu	0	0.0
Ram	0	0.0
Memory	0	0.0
Gpu	0	0.0
OpSys	0	0.0
Weight	0	0.0
Price	0	0.0
Duplicate Rows:	0	

Q4. Statistical Summary)

- Use describe () to generate statistical summary of numeric columns.
- Identify min, max, mean, and median of the target variable.
- Write at least 4 meaningful observations about the data distribution.

Answer:

- Minimum Price = 9270.72
- Maximum Price = 324954.72
- Mean Price ≈ 59870.04

- Median Price $\approx$ 52054.56

PROGRAM SCREENSHOT :

```
#Q4

print(df.describe())

print("Minimum Price :",df["Price"].min())
print("Maximum Price :",df["Price"].max())
print("Mean Price :",df["Price"].mean())
print("Median Price :",df["Price"].median())
```

	Unnamed: 0	Inches	Price
count	1303.00000	1303.000000	1303.000000
mean	651.00000	15.017191	59870.042910
std	376.28801	1.426304	37243.201786
min	0.00000	10.100000	9270.720000
25%	325.50000	14.000000	31914.720000
50%	651.00000	15.600000	52054.560000
75%	976.50000	15.600000	79274.246400
max	1302.00000	18.400000	324954.720000
Minimum Price : 9270.72			
Maximum Price : 324954.72			
Mean Price : 59870.042910207216			
Median Price : 52054.56			

Q5. Histogram Analysis of Numeric Features)

- Plot histograms for all important numeric columns (including the target variable).
- Write detailed insights for each plot.

Answer:

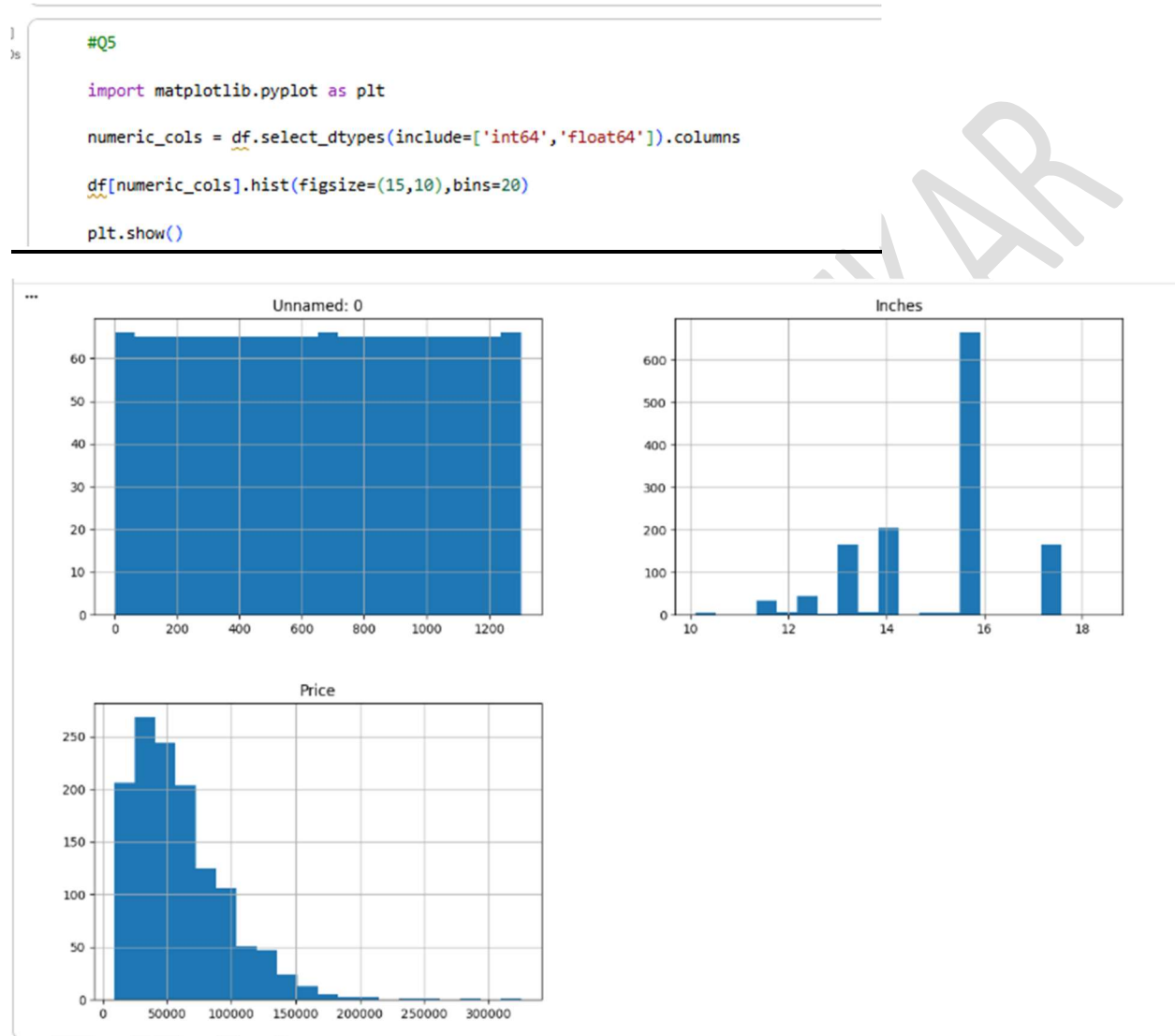
Price Histogram

- Most laptops cost between ₹30,000–₹80,000.
- Distribution is positively skewed.
- Few expensive laptops act as outliers.

Inches Histogram

- Most laptops have 14– 15.6 inch screens.
- Small and large screen laptops are fewer.

PROGRAM SCREENSHOT :



Q6. Count Plots of Categorical Features)

- Identify categorical columns in your dataset.
- Create count plots for all categorical features using seaborn.
- Analyze the distribution and write business/domain-related insights

Answer:

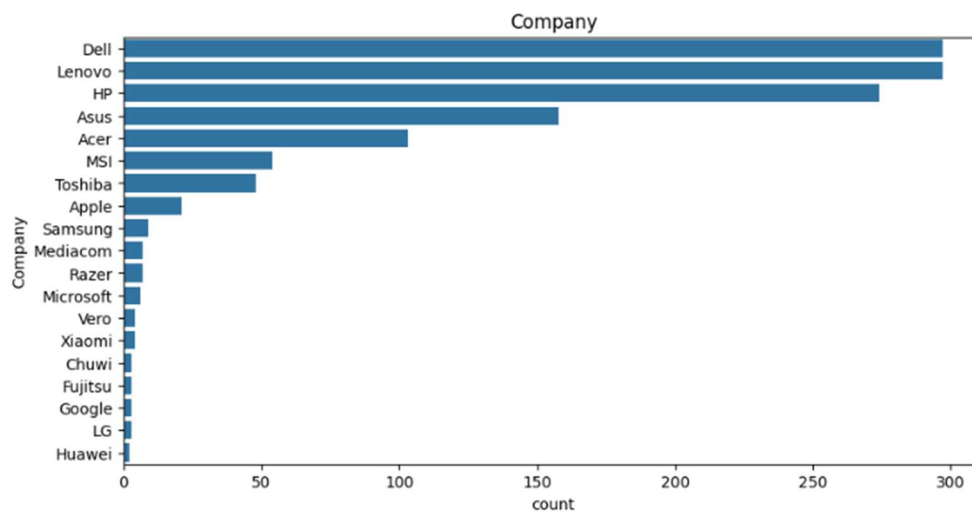
- Lenovo, HP, Dell, and Asus are the most common brands.
- Notebook is the most common laptop type.
- Windows OS dominates the dataset.
- 8GB RAM laptops are the most common.

PROGRAM SCREENSHOT :

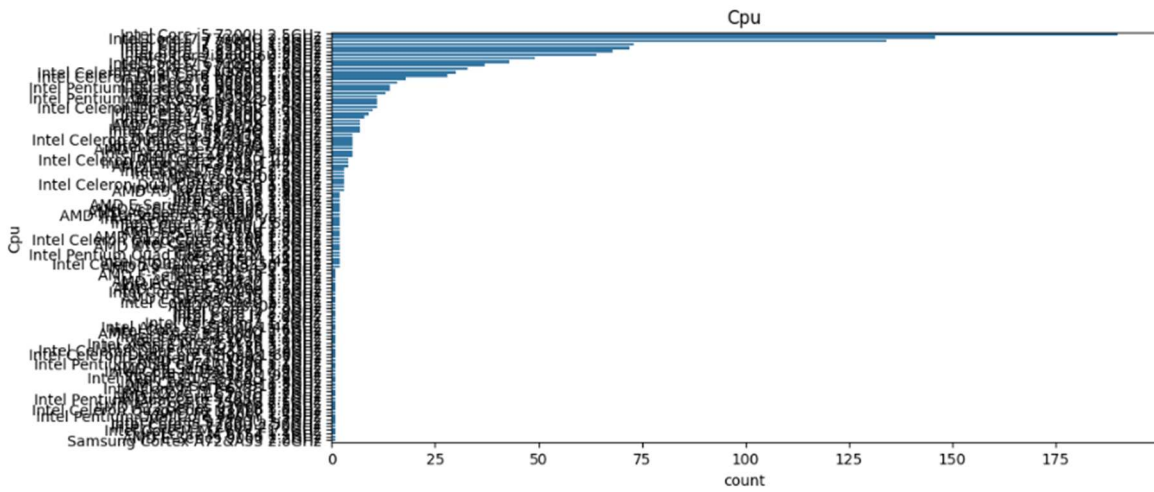
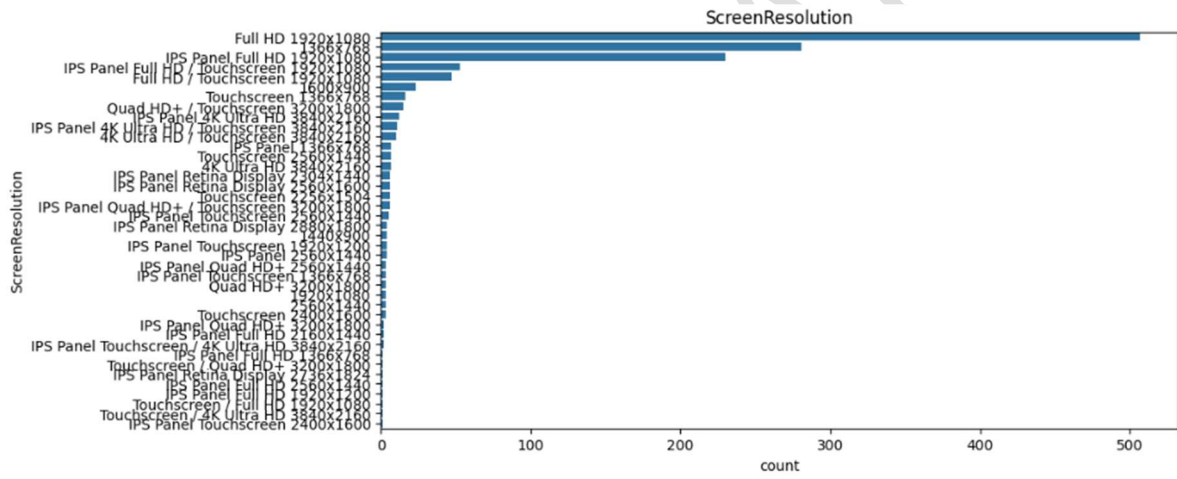
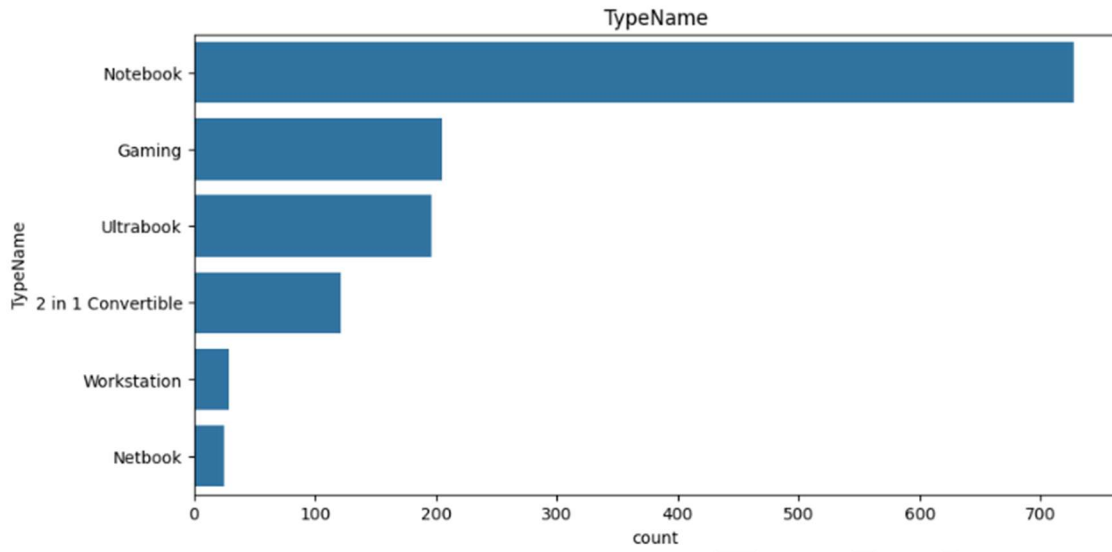
```
#Q6
import seaborn as sns

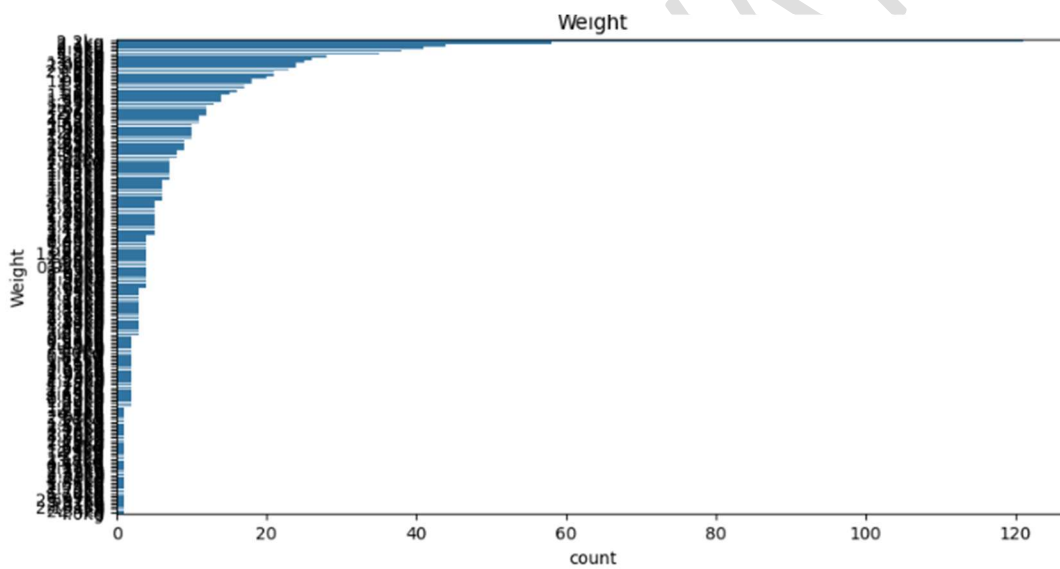
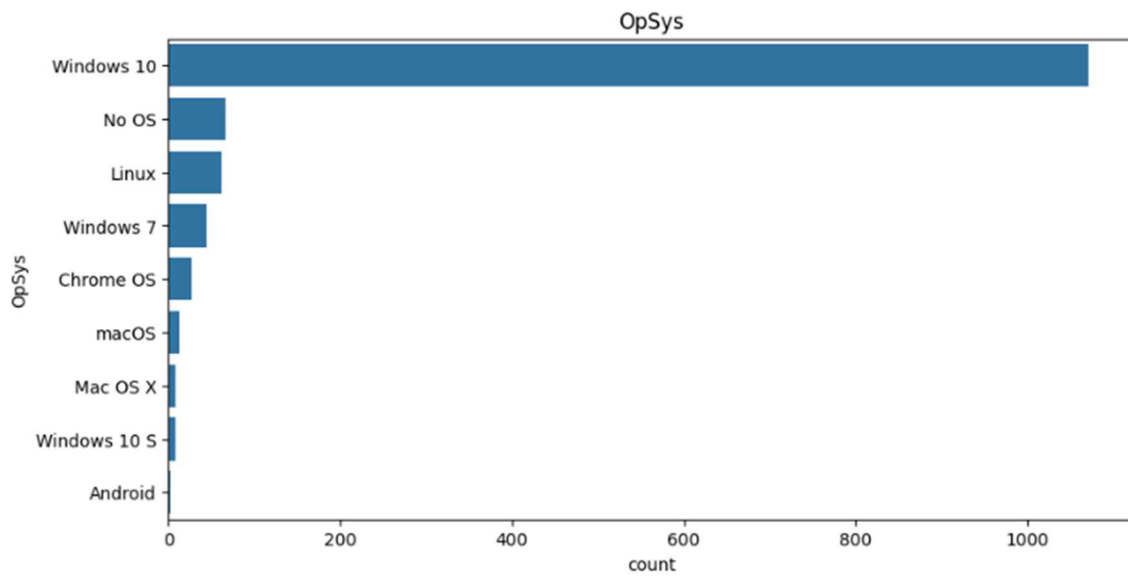
categorical_cols = df.select_dtypes(include='object').columns

for col in categorical_cols:
    plt.figure(figsize=(10,5))
    sns.countplot(y=df[col], order=df[col].value_counts().index)
    plt.title(col)
    plt.show()
```



count





Q7. Correlation Heatmap)

- Create a correlation heatmap for all numeric features.
- Highlight which features are strongly correlated with the target variable.
- Write your observations regarding feature relationships

Answer:

- Price has a positive correlation with screen size (Inches).
- Other numeric features show weak correlation before encoding.
- Stronger relationships appear after encoding categorical features.

PROGRAM SCREENSHOT :

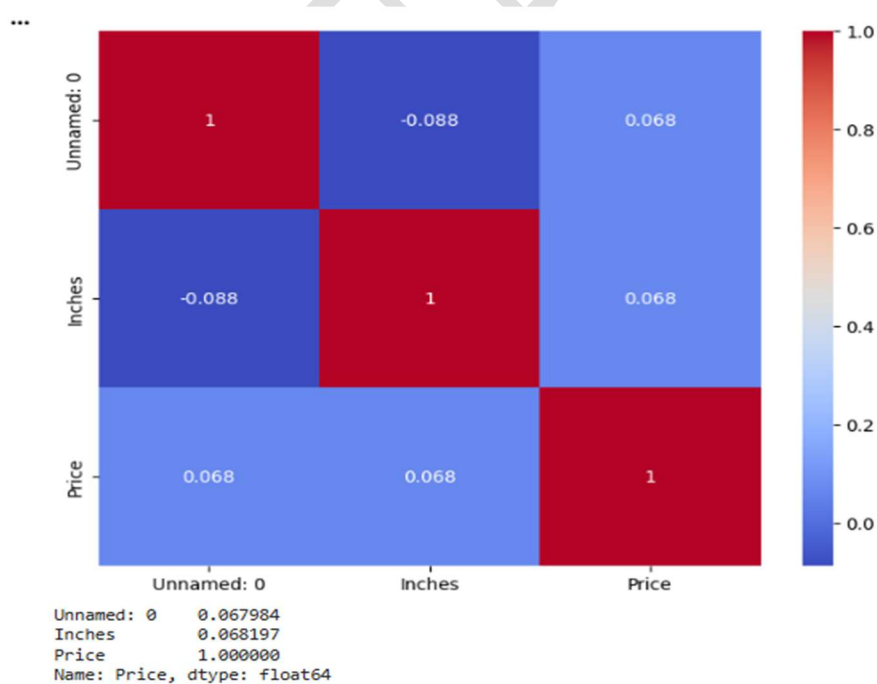
```
#Q7
numeric_df = df.select_dtypes(include=['int64', 'float64'])

plt.figure(figsize=(8,6))

sns.heatmap(numeric_df.corr(),
            annot=True,
            cmap='coolwarm')

plt.show()

print(numeric_df.corr()["Price"])
```



Q8. Feature Identification) Clearly define:

- Independent Features X — input features
- Dependent Feature (y) — continuous target variable you want to predict
- Justify your selection with reasoning.

Answer:

Independent Features (X)

- Company
- TypeName
- Inches
- Screen Resolution
- Cpu
- Ram
- Memory
- Gpu
- OpSys
- Weight

Dependent Feature (y)

- Price

Reason: Price depends on all laptop specifications.

PROGRAM SCREENSHOT :

```
#Q8

X = df.drop("Price",axis=1)

y = df["Price"]

print("Independent Features")
print(X.columns)

print("Dependent Feature")
print(y.name)

*** Independent Features
Index(['Unnamed: 0', 'Company', 'TypeName', 'Inches', 'ScreenResolution',
      'Cpu', 'Ram', 'Memory', 'Gpu', 'OpSys', 'Weight'],
      dtype='object')
Dependent Feature
Price
```

Q9. Encoding Categorical Variables)

Convert all categorical columns to numeric form using suitable encoding techniques One-Hot Encoding).

- Show before and after transformation for at least two columns.
- Explain your encoding choices.

Answer:

One-Hot Encoding converts categorical values into numerical values.

Example:

- Before

Company	TypeName
Apple	Ultrabook
HP	Notebook

- After

Company_HP	Company_Lenovo	Type Name_Notebook	TypeName_Ultrabook
0	0	0	1
1	0	1	0

V

PROGRAM SCREENSHOT :

```

#Q9
print("Before Encoding")

print(df[['Company', 'TypeName']].head())

df_encoded = pd.get_dummies(df, drop_first=True)

print("After Encoding")

print(df_encoded.head())

```

```

... Before Encoding
    Company  TypeName
0   Apple  Ultrabook
1   Apple  Ultrabook
2     HP   Notebook
3   Apple  Ultrabook
4   Apple  Ultrabook
After Encoding
  Unnamed: 0  Inches      Price  Company_Apple  Company_Asus  \
0           0   13.3    71378.6832             True      False
1           1   13.3    47895.5232             True      False
2           2   15.6    30636.0000             False     False
3           3   15.4   135195.3360             True      False
4           4   13.3    96095.8080             True      False

    Company_Chuiwi  Company_Dell  Company_Fujitsu  Company_Google  Company_HP  \
0           False      False      False      False      False
1           False      False      False      False      False
2           False      False      False      False      True
3           False      False      False      False      False
4           False      False      False      False      False

... Weight_4.2kg  Weight_4.33kg  Weight_4.36kg  Weight_4.3kg  \
0 ...           False      False      False      False
1 ...           False      False      False      False
2 ...           False      False      False      False
3 ...           False      False      False      False
4 ...           False      False      False      False

    Weight_4.42kg  Weight_4.4kg  Weight_4.5kg  Weight_4.6kg  Weight_4.7kg  \
0           False      False      False      False      False
1           False      False      False      False      False
2           False      False      False      False      False
3           False      False      False      False      False
4           False      False      False      False      False

    Weight_4kg
0           False
1           False
2           False
3           False
4           False

[5 rows x 523 columns]

```

Q10. Feature Scaling) Apply Standard Scaling on the independent features using Standard Scaler.

- Show the first 5 rows of the scaled features.

Answer:

- Standard Scaler standardizes all features.
- Mean becomes approximately 0.
- Standard deviation becomes 1.
- Scaling improves the performance of machine learning algorithms by ensuring features are on a similar scale.

PROGRAM SCREENSHOT :

```
#Q10
from sklearn.preprocessing import StandardScaler

X = df_encoded.drop("Price",axis=1)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

scaled_df = pd.DataFrame(X_scaled,columns=X.columns)

print(scaled_df.head())
```



```

...   Unnamed: 0   Inches   Company_Apple   Company_Asus   Company_Chui \
0   -1.730722   -1.204407   7.813298   -0.371472   -0.048038
1   -1.728063   -1.204407   7.813298   -0.371472   -0.048038
2   -1.725405   0.408772   -0.127987   -0.371472   -0.048038
3   -1.722746   0.268495   7.813298   -0.371472   -0.048038
4   -1.720088   -1.204407   7.813298   -0.371472   -0.048038

      Company_Dell   Company_Fujitsu   Company_Google   Company_HP   Company_Huawei \
0   -0.543349   -0.048038   -0.048038   -0.516021   -0.039208
1   -0.543349   -0.048038   -0.048038   -0.516021   -0.039208
2   -0.543349   -0.048038   -0.048038   1.937905   -0.039208
3   -0.543349   -0.048038   -0.048038   -0.516021   -0.039208
4   -0.543349   -0.048038   -0.048038   -0.516021   -0.039208

...   Weight_4.2kg   Weight_4.33kg   Weight_4.36kg   Weight_4.3kg \
0   ...   -0.048038   -0.027714   -0.055491   -0.055491
1   ...   -0.048038   -0.027714   -0.055491   -0.055491
2   ...   -0.048038   -0.027714   -0.055491   -0.055491
3   ...   -0.048038   -0.027714   -0.055491   -0.055491
4   ...   -0.048038   -0.027714   -0.055491   -0.055491

      Weight_4.42kg   Weight_4.44kg   Weight_4.5kg   Weight_4.6kg   Weight_4.7kg \
0   -0.092271   -0.027714   -0.027714   -0.055491   -0.027714
1   -0.092271   -0.027714   -0.027714   -0.055491   -0.027714
2   -0.092271   -0.027714   -0.027714   -0.055491   -0.027714
3   -0.092271   -0.027714   -0.027714   -0.055491   -0.027714
4   -0.092271   -0.027714   -0.027714   -0.055491   -0.027714

      Weight_4kg
0   -0.027714
1   -0.027714
2   -0.027714
3   -0.027714
4   -0.027714

[5 rows x 522 columns]

```